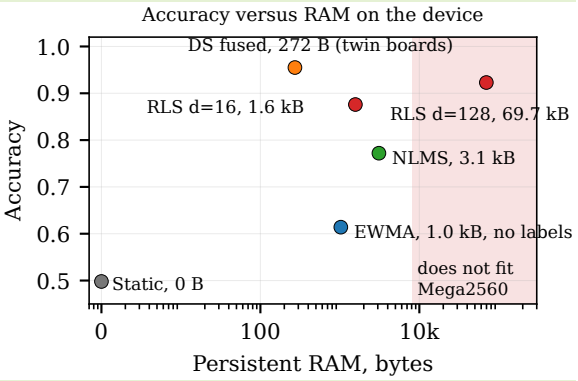
Memory and Time Costs of Drift Compensation and Calibration Transfer for Electronic Noses on Microcontrollers

Oleksandr Kuznetsov, *Member, IEEE*

***Abstract*— Like its biological prototype, an electronic nose relies on cross-sensitive receptors whose responses drift over time and differ between arrays. Compensation methods are usually compared by accuracy alone, although a low-cost instrument must run them on a microcontroller with a few kilobytes of RAM. We measure the RAM and execution time of linear drift-compensation and calibration-transfer methods that adapt a least-squares (ridge) linear classifier, on two public metal-oxide array datasets (36-month drift; five twin boards) and on Mega2560, ESP32-C3 and ESP32-S3 boards. Without labels, running standardisation raises accuracy on the drift stream from 0.498 to 0.614 with 1 028 B of RAM, an improvement seen in seven of eight test batches. With 20 references per batch drawn from all gases, recursive least squares reaches 0.923 but keeps 69.7 kB in RAM, whereas on 16 compact features it reaches 0.876 with 1.6 kB; when all references of a session come from one gas, all label-driven updates fall below the unadapted classifier. Between twin boards, direct standardisation fused into the classifier reaches 0.955 with 272 B kept in RAM and a 4.6 kB peak during the fit, which takes 0.27 s on the Mega2560. Float32 C kernels reproduce 12.3 million host predictions to within 65 and the feature extractor is bit-identical on all 640 complete recordings. For slow gas sensing, memory rather than time decides what an 8-bit device can run.**



***Index Terms*— Calibration transfer, drift compensation, electronic nose, embedded systems, gas sensor arrays, microcontrollers, recursive least squares.**

I. Introduction

THE electronic nose was proposed as a model of the mammalian olfactory system: instead of one selective receptor per odorant, an array of broadly tuned sensors produces a response pattern that is recognised by a trained classifier [1]. Metal-oxide (MOX) gas sensors are the most common receptors in such arrays because they are cheap, sensitive and easy to read out. Their weakness is equally well known. The response of a MOX sensor changes over months because of ageing, poisoning and environmental history, and two nominally identical arrays respond differently to the same gas [2]. A classifier trained once, therefore, gradually loses accuracy on the same instrument, and a classifier trained on one instrument does not transfer to another without correction.

Both problems have been studied extensively. For long-term drift, component correction [3], orthogonal signal correction [4], common principal components [5], classifier ensembles [6] and domain-adaptation extreme learning machines [7] have been proposed, and a public 36-month dataset [6], [8] has become the common benchmark. Since every labelled sample costs a calibration measurement, later work has reduced the number of labels by active learning on dynamic clusters [13], by online drift compensation on large response streams [14] and by cross-domain active learning [15]. For instrument-to-instrument variation, direct standardisation (DS), originally developed for spectrometers [9], has been applied to twin MOX arrays with only a few transfer samples [10]. In this literature, however, the methods run on a workstation and the criterion is accuracy, sometimes together with the number of labels; the memory and time that a method needs on the instrument itself are not reported. A recent review of drift compensation for MOX arrays names the trade-off between performance and efficiency, and models that can actually be deployed, among the open problems [11]. The closest embedded work we found trains a compact neural network for drift compensation offline and quantizes it for inference [12]; on-device adaptation, its memory and its execution time on a microcontroller are not measured there, and we found no study that does so for electronic noses.

For a low-cost electronic nose, this omission matters. An

O. Kuznetsov is with the Department of Theoretical and Applied Sciences, eCampus University, Italy, and with the Department of Intelligent Software Systems and Technologies, School of Computer Science and Artificial Intelligence, V.N. Karazin Kharkiv National University, Ukraine (e-mail: oleksandr.kuznetsov@uniecampus.it; ORCID: 0000-0003-2331-6326).

Arduino-class 8-bit microcontroller has 8 kB of SRAM, and even ESP32-class devices share a few hundred kilobytes with communication stacks. A compensation method that updates its model from reference measurements must keep the model and its own state in RAM between samples, and it may need further temporary memory when a calibration session ends. Studies of on-device learning make the same observation for neural networks: update memory must be counted separately from inference memory [16]. At the same time, the timing requirement of a gas sensor array is mild, because one measurement cycle lasts minutes. It is therefore not obvious in advance whether memory or time limits the choice of method on a given processor.

The aim of this work is to quantify, on public recordings of real MOX arrays and on physical microcontrollers, the accuracy, the RAM and the execution time of simple drift-compensation and calibration-transfer methods. We deliberately restrict ourselves to methods that adapt a linear least-squares classifier or a linear feature map, because they can be written as short float32 kernels whose memory is known exactly, and we do not compete with ensembles or deep models on accuracy. The contribution of the paper is as follows.

1) We compare eight compensation methods and two combinations on the 36-month drift dataset, and eight methods including three calibration-transfer methods on five twin boards, under a protocol whose splits, hyperparameter grids and seeds were fixed before the experiments, and we account for the RAM of each method between samples and at the end of a calibration session.
2) We implement the methods that matter for deployment, together with a streaming feature extractor, as float32 C kernels, show that they reproduce the host on complete data streams, and run them on three boards (AVR, RISC-V and Xtensa with a floating-point unit).
3) We report the measured time per call on the three boards and show where the results depend on favourable calibration conditions, which leads to a deployment map in which memory rather than time decides what can run on an 8-bit device.

II. Data and Common Model

A. Datasets

We use two public datasets of the UCI repository. The first (D1) is the gas sensor array drift dataset [6], [8], [17], in the version with concentrations [18]. It contains 13 910 recordings of six gases (ammonia, acetaldehyde, acetone, ethylene, ethanol and toluene) measured by an array of 16 MOX sensors over 36 months and grouped into ten batches. Each recording is described by 128 features, eight per sensor: the steady-state resistance change ΔR, its normalised form, and the maxima and minima of an exponential moving average of the resistance derivative for three smoothing constants, as introduced in [19]. Batches are unbalanced (161 to 3 613 recordings), and toluene is absent from batches 3–5.

The second dataset (D2) contains recordings of five twin arrays of eight Figaro MOX sensors exposed to carbon monoxide, ethanol, ethylene and methane at ten concentration levels [10], [20]. Its 640 raw recordings last 600 s at 100 Hz. According to the dataset description, boards 1–3 were measured on four days and boards 4 and 5 on two; the description lists the days but not which repetition index belongs to which day, and we assume that repetitions follow the order of the days. Only the day-to-day analysis of E4 depends on this assumption. Nine recordings have missing samples (fewer than 50 000 of the nominal 60 000), and we check the effect of these and of the methane recordings of board 5 in Section V-E. All archives were verified by SHA-256 digests before use.

B. Features and classifier

For D1 we use either all 128 features ($d = 128$) or the 16 ΔR features ($d = 16$). For D2, the features are computed by our own streaming extractor (Section III-A), and we use either all eight features of each sensor ($d = 64$) or only ΔR and $\Delta R/R_0$ ($d = 16$). Each feature is compressed by the signed logarithm $s(x) = \operatorname{sign}(x)\log(1 + |x|)$ and standardised with the mean and standard deviation of the source data, giving the vector $\boldsymbol{z} \in \mathbb{R}^d$.

All methods share one linear classifier. With $\boldsymbol{a} = [\boldsymbol{z}^T, 1]^T$, the predicted class is

$$\hat{c} = \arg\max_c \; \boldsymbol{a}^T \boldsymbol{w}_c, \qquad \boldsymbol{W} = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_C] \in \mathbb{R}^{(d+1)\times C}. \tag{1}$$

The head $\boldsymbol{W}$ is fitted by ridge regression of the n source samples $\boldsymbol{A} \in \mathbb{R}^{n\times(d+1)}$ onto one-hot class targets $\boldsymbol{T}$,

$$\boldsymbol{W} = (\boldsymbol{A}^T\boldsymbol{A} + \boldsymbol{\Lambda})^{-1}\boldsymbol{A}^T\boldsymbol{T}, \qquad \boldsymbol{\Lambda} = \operatorname{diag}(\lambda, \ldots, \lambda, \epsilon_0), \tag{2}$$

where the intercept receives only the numerical penalty $\epsilon_0 = 10^{-9}$; the same convention is used in every ridge fit below. Such a least-squares classifier is weaker than a support-vector machine, but it has an important advantage for this study: the sequential and block estimators described below update exactly the model that is deployed. In float32, the head occupies $4C(d+1)$ bytes, i.e., 408 B for $d = 16$ and 3 096 B for $d = 128$ in D1 ($C = 6$), and 272 B for $d = 16$ in D2 ($C = 4$). For concentration regression (Section IV), one output per gas predicts $\log_{10}$ of the concentration in ppmv.

III. Compensation Methods and Memory Accounting

We distinguish four kinds of RAM. The *state* of a method is what it keeps between samples in addition to the head, counted in float32 values and one byte per stored class label. The *persistent RAM* adds the head whenever the method modifies it; methods that leave the head unchanged can keep it in flash memory. *Scratch* memory holds the feature vector of the current sample and intermediate vectors of one call. The *calibration peak* is the temporary memory needed when a calibration session ends, for example to solve a least-squares problem. Table VII gives all of them for the configurations implemented in the firmware.

M0, static head. The source model is applied without change; it has no state.

M1, EWMA standardisation. Without any labels, the running mean $\boldsymbol{m}$ and variance $\boldsymbol{v}$ of the incoming features replace the source statistics:

$$\boldsymbol{m}_t = \boldsymbol{m}_{t-1} + \alpha(\boldsymbol{z}_t - \boldsymbol{m}_{t-1}), \tag{3}$$

$$\boldsymbol{v}_t = (1-\alpha)\big(\boldsymbol{v}_{t-1} + \alpha(\boldsymbol{z}_t - \boldsymbol{m}_{t-1})^{\odot 2}\big), \tag{4}$$

and the head receives $(\boldsymbol{z}_t - \boldsymbol{m}_t) \oslash (\boldsymbol{v}_t + \epsilon)^{\odot 1/2}$. The method resembles the adaptation of a receptor to a constant background, and its state is $8d + 4$ bytes.

M2–M4, other feature-level corrections. Adaptive class centroids updated without labels (M2, $4Cd + 4$ B), correction along r drift directions estimated from the references in the spirit of component correction [3] (M3, $4rd + 4$ B), and a per-feature gain and offset fitted to the references (M4, $8d$ B) complete the set.

M5, NLMS on the head. For each reference $(\boldsymbol{z}, c)$ with one-hot target $\boldsymbol{t}$, the head is updated as [21]

$$\boldsymbol{W} \leftarrow \boldsymbol{W} + \frac{\mu}{\|\boldsymbol{a}\|^2 + \epsilon}\, \boldsymbol{a}\,(\boldsymbol{t} - \boldsymbol{W}^T\boldsymbol{a})^T. \tag{5}$$

Only the step size is stored (4 B), but the head must be in RAM.

M6, RLS on the head. Exponentially weighted RLS keeps the $(d+1) \times (d+1)$ inverse correlation matrix $\boldsymbol{P}$, initialised from the source fit as $\boldsymbol{P}_0 = (\boldsymbol{A}^T\boldsymbol{A} + \boldsymbol{\Lambda})^{-1}$ with $\boldsymbol{\Lambda}$ of (2), so that the first update continues the ridge solution instead of restarting it. With forgetting factor β,

$$\boldsymbol{g} = \frac{\boldsymbol{P}\boldsymbol{a}}{\beta + \boldsymbol{a}^T\boldsymbol{P}\boldsymbol{a}}, \qquad \boldsymbol{W} \leftarrow \boldsymbol{W} + \boldsymbol{g}(\boldsymbol{t} - \boldsymbol{W}^T\boldsymbol{a})^T, \tag{6}$$

$$\boldsymbol{P} \leftarrow \beta^{-1}(\boldsymbol{P} - \boldsymbol{g}\boldsymbol{a}^T\boldsymbol{P}). \tag{7}$$

The state is $4(d+1)^2 + 4$ B: 1 160 B for $d = 16$, but 66 568 B for $d = 128$.

M7, block least squares. The k references of one calibration session are buffered, and the head is refitted once by ridge regression shrunk towards the current head,

$$\boldsymbol{W} \leftarrow \arg\min_{\boldsymbol{X}} \ \|\boldsymbol{T}_k - \boldsymbol{A}_k\boldsymbol{X}\|_F^2 + \mathrm{tr}\big((\boldsymbol{X} - \boldsymbol{W})^T\boldsymbol{\Lambda}_\gamma(\boldsymbol{X} - \boldsymbol{W})\big), \tag{8}$$

with $\boldsymbol{\Lambda}_\gamma = \mathrm{diag}(\gamma, \ldots, \gamma, \epsilon_0)$, which is solved by a Cholesky factorisation of a $(d+1) \times (d+1)$ matrix. The buffer takes $k(4d+1)$ B, and the fit needs the Gram matrix, the solution and the targets as temporary memory.

M8 and M9, direct standardisation. For calibration transfer, k pairs of feature vectors of the same gas and concentration, measured on the master and on the slave board, give the affine map $\boldsymbol{F} \in \mathbb{R}^{(d+1)\times d}$ from slave to master features, $\boldsymbol{z}_\mathrm{m} \approx \boldsymbol{F}^T\boldsymbol{a}_\mathrm{s}$. We fit it by (8) with the master features as targets and the prior $[\boldsymbol{I}, \boldsymbol{0}]^T$ in place of $\boldsymbol{W}$. Stored separately, the map takes $4d(d+1)$ B (1 088 B for $d = 16$). Since the head is linear, the map can be fused into it once after the fit,

$$\boldsymbol{W}' = \boldsymbol{F}\,\boldsymbol{W}_{1:d} + \boldsymbol{e}_{d+1}\boldsymbol{w}_{d+1}^T, \tag{9}$$

where $\boldsymbol{W}_{1:d}$ are the first d rows of $\boldsymbol{W}$ and $\boldsymbol{w}_{d+1}^T$ is its intercept row, so that $\boldsymbol{a}_\mathrm{s}^T\boldsymbol{W}' = [\boldsymbol{z}_\mathrm{m}^T, 1]\boldsymbol{W}$. The fused head occupies $4C(d+1)$ B (272 B for $d = 16$) and is applied at the cost of the static head. The per-sensor variant M9 is block diagonal: one affine map of the q features of each sensor ($q = 2$ for $d = 16$, $q = 8$ for $d = 64$), fitted in the same way.

Two combinations, M1+M5 and M1+M7, apply the labelled update after the unsupervised standardisation.

A. Streaming feature extractor

The D1 features were computed by the dataset authors from complete recordings. For D2 we compute features of the same kind from the raw resistance traces with an extractor that processes one sample at a time and keeps a constant state, as it would on an instrument. During the first 30 s, before the gas reaches the chamber, it accumulates the baseline R_0 of each sensor; afterwards it tracks the minimum resistance R_min. For each of three smoothing constants $\alpha \in \{0.001, 0.01, 0.1\}$, an exponential moving average of the first difference of the resistance is updated, and its maximum and minimum are kept. At the end of a recording, the eight features of each sensor are $\Delta R = R_0 - R_\mathrm{min}$, $\Delta R / R_0$ and the six extrema. For eight sensors, the state takes 426 B on the Mega2560 and 428 B on the ESP32 boards, which differ only in structure padding.

IV. Protocol

The protocol of E1–E5, including data splits, hyperparameter grids, selection rules and random seeds, was written down and committed to the public repository before each experiment was run (E1/E2 and E3/E4 separately); results were not used to change it. Hyperparameters were selected only on D1 batch 2 and on the D2 board pair $1 \to 2$, which are excluded from the reported averages. The analyses E6 and E7 were added after an internal review of the first draft; they change no frozen setting, are reported as post hoc, and are logged with their reasons in the repository.

E1, static settings. The head is trained on batch 1 and tested on batches 2–10 (setting 1), or trained on batch $b-1$ and tested on batch b (setting 2); a multinomial logistic head is included for reference.

E2, streaming drift compensation (main experiment). Batches 2–10 of D1 are processed in order, and the state of every method carries over from batch to batch. In each batch, a seeded permutation fixes the sample order; the first $k \in \{0, 1, 5, 20\}$ samples are labelled references, i.e., one calibration session, and the remaining samples are classified in order without labels. The references are thus drawn uniformly from all gases present in the batch, which is favourable; E6 relaxes this. Accuracy and macro-F1 are averaged over batches 3–10 and 20 permutation seeds. The head ridge parameter and the method hyperparameters were selected from fixed grids by the mean accuracy on batch 2 with five separate tuning seeds and $k = 5$.

E3, concentration under drift. The gas identity is assumed to be known, and one regression head per gas predicts $\log_{10}$ ppmv. The stream contains only the recordings of that gas, so k references per batch are k references of that gas; budgets and selection otherwise follow E2, and the metric is the root-mean-square error (RMSE) averaged over gases.

E4, calibration transfer and day-to-day drift. The head is trained on all recordings of a master board and applied to a slave board, for all 20 ordered pairs. For $k \in$

$\{0, 2, 4, 8, 16, 40\}$, k of the 40 gas–level conditions are drawn at random per seed, and their first-day recordings on both boards form transfer pairs (M4, M8, M9) or labelled references (M5–M7). All other recordings of the slave board are evaluated; M1 uses no labels, and its results at $k > 0$ are computed on the same evaluation records as the other methods. We report the mean and the worst of the 19 pairs other than the tuning pair, over 20 seeds. Within boards 1–3, we also train on the first day and test on the following ones.

E5, microcontrollers. The kernels of Section III and the feature extractor were written in C and compiled with `-fno-fast-math -ffp-contract=off` both for the host and for the boards. A fixture of real samples is stored in flash: 64 and 32 recordings of D1 batch 7 for $d = 16$ and $d = 128$ with the frozen E2 heads and hyperparameters, 20 references and 44 test recordings for the block refit, 16 transfer pairs and 64 test recordings from D2 boards $1 \to 2$ for DS, and 512 samples of one raw D2 recording for the extractor. Each kernel first runs once and is checked against the host reference output; only a kernel that passes is timed ten times with `micros()`. Each board was flashed once and measured in two sessions. Table I lists the boards and builds.

E6, post hoc host analyses. (i) All E2 streams of M0, M1, M5–M7 and the two combinations, and all E4 runs of DS, were repeated with the float32 C kernels of the firmware and compared prediction by prediction with the float64 host; for DS, the fused head (9) was compared as well. The extractor was run over all 640 complete D2 recordings. (ii) E2 was repeated with the recordings of each batch arriving in blocks of one gas, and with all k references of a session drawn from one gas. (iii) Implementation variants: block least squares with the intercept penalised like the weights (M7b), RLS with the trace of $\boldsymbol{P}$ bounded by that of $\boldsymbol{P}_0$ (M6b), and RLS with forgetting factors 0.95 and 0.99 in E2 and E3. (iv) E4 without the 20 methane recordings of board 5 and without the nine recordings with missing samples.

E7, complete streams on the boards. A second firmware receives real samples over USB and runs complete E2 streams (seed 0, $k = 20$), DS with the fit and the fusion on the board, and complete 600 s D2 recordings through the extractor, while the peak stack depth is recorded (stack painting on the Mega2560, the high-water mark of the loop task on the ESP32 boards).

Statistics. The eight test batches of E2 form one stream whose state carries over, and the 19 board pairs of E4 share boards and recordings, so neither can be treated as independent units. For E2 we therefore report differences per batch (mean, number of batches in favour and range); intervals over the 20 reference draws are narrow and describe only this one stream. For E4 we use a cluster bootstrap that resamples the five boards with replacement and averages the differences over all ordered pairs of distinct drawn boards (10 000 replicates) [22]. Every comparison is made on identical test records.

TABLE I
BOARDS, TOOLCHAINS AND FIRMWARE SIZE (E5)

Board	Core, clock	FPU	RAM used (B)	Flash used (B)
Mega2560	AVR, 16 MHz	no	5 497 / 8 192	67 378
ESP32-C3	RV32IMC, 160 MHz	no	84 812 / 327 680	348 758
ESP32-S3	Xtensa LX7, 240 MHz	single	89 412 / 327 680	362 101

Arduino framework via PlatformIO; avr-gcc 7.3.0 for Mega2560, GCC 8.4.0 (esp-2021r2-patch5) for the ESP32 boards, default `-Os`. RAM includes a static arena of 4 600 B on Mega2560 and 70 724 B on the ESP32 boards.

V. RESULTS ON THE HOST

A. Drift over 36 months

In the static settings of E1, a head trained on batch 1 reaches only 0.542 on batches 2–10 with all 128 features and 0.468 with 16 features, whereas retraining on the previous batch gives 0.783 and 0.606. Retraining, however, needs the labels of a whole batch and either the batch itself (up to 3 613 recordings, about 1.9 MB at $d = 128$) or its normal equations $(4(d+1)(d+1+C)$ B, i.e., 69.7 kB at $d = 128$ and 1.6 kB at $d = 16$), so it serves only as a frame for the streaming results.

Table II and Fig. 1(a) summarise E2, and Table III gives the paired differences. Without any labels, EWMA standardisation improves the static head from 0.434 to 0.541 at $d = 16$ (132 B) and from 0.498 to 0.614 at $d = 128$ (1 028 B), and macro-F1 from 0.346 to 0.435 and from 0.387 to 0.560. The gain is not uniform: it is positive in six and seven of the eight test batches, and in the remaining batches EWMA loses up to 0.095. Adaptive centroids lower the accuracy (0.302 and 0.436), and the per-feature affine correction stays near the static head (0.465 and 0.464 at $k = 20$). Component correction, in contrast, profits from the references at $d = 128$ and reaches 0.658 at $k = 20$ with 1 540 B, above EWMA, while at $d = 16$ it reaches 0.503.

Labels change the picture more strongly. With 20 references per batch, NLMS at $d = 128$ reaches 0.772 and is better than the static head in all eight batches; it keeps only 4 B of state, but the modified head adds 3 096 B. RLS reaches the highest accuracy, 0.923, and exceeds NLMS in every batch, but together with the head its covariance matrix keeps 69.7 kB in RAM, more than eight times the SRAM of the Mega2560. On the 16 compact features, the same RLS keeps 1.6 kB and reaches 0.876, and block least squares reaches 0.898 with 1.7 kB and a 3.8 kB peak at the refit. The two are close: block least squares is better in four of the eight batches, by 0.022 on average. One reference per batch is not sufficient, and block least squares even collapses to 0.243 at $d = 16$. Two causes contribute. Its shrinkage $\gamma = 0.1$ was selected at $k = 5$, and with a single one-hot target the almost unpenalised intercept moves freely towards that class. Penalising the intercept like the weights (M7b, post hoc) raises the result to 0.313 at $d = 16$ and from 0.481 to 0.590 at $d = 128$, still below the static head at $d = 16$; with $k \geq 5$ the variant changes the result by at most 0.011.

B. Less favourable calibration

The references of E2 come from all gases of a batch, and the recordings arrive in random order. Table IV relaxes both conditions. When the recordings of a batch arrive in blocks of one gas, EWMA standardisation suffers most, because its running statistics follow the current gas: at $d = 128$ its accuracy falls from 0.614 to 0.541. The label-driven methods

TABLE II

DRIFT COMPENSATION ON THE GAS SENSOR ARRAY DRIFT DATA: ACCURACY (BATCHES 3–10, 20 SEEDS), MACRO-F1 AT $k = 20$ AND PERSISTENT RAM

	$d = 16$						$d = 128$					
Method	$k = 0$	1	5	20	$F1_{20}$	RAM (B)	$k = 0$	1	5	20	$F1_{20}$	RAM (B)
M0 Static head	0.434	0.434	0.434	0.433	0.345	0	0.498	0.498	0.498	0.498	0.388	0
M1 EWMA standardisation	0.541	0.541	0.541	0.541	0.436	132	0.614	0.614	0.614	0.614	0.561	1 028
M2 Adaptive centroids	0.302	0.302	0.302	0.301	0.222	388	0.435	0.435	0.436	0.439	0.333	3 076
M3 Component correction	–	0.464	0.438	0.503	0.386	196	–	0.479	0.585	0.658	0.565	1 540
M4 Per-feature affine	–	0.476	0.430	0.465	0.353	128	–	0.511	0.428	0.464	0.373	1 024
M5 NLMS on head	–	0.447	0.479	0.574	0.466	412	–	0.579	0.676	0.772	0.678	3 100
M6 RLS on head	–	0.447	0.631	0.876	0.780	1 568	–	0.634	0.792	0.923	0.850	69 664
M7 Block OLS on head	–	0.243	0.614	0.898	0.799	1 708	–	0.481	0.721	0.855	0.770	13 356
M1+M5 EWMA + NLMS	–	0.546	0.557	0.575	0.468	544	–	0.645	0.710	0.798	0.712	4 128
M1+M7 EWMA + block OLS	–	0.246	0.591	0.842	0.741	1 840	–	0.503	0.671	0.803	0.705	14 384

k: labelled references per batch (one calibration session). $F1_{20}$: macro-F1 at $k = 20$. RAM: bytes kept between samples in float32, i.e., the adaptation state plus the head (408 B at $d = 16$, 3 096 B at $d = 128$) for methods that modify it (M5–M7 and combinations); other heads can stay in flash. Unsupervised methods (M1, M2) do not use the references. M3 and M4 need at least one reference.

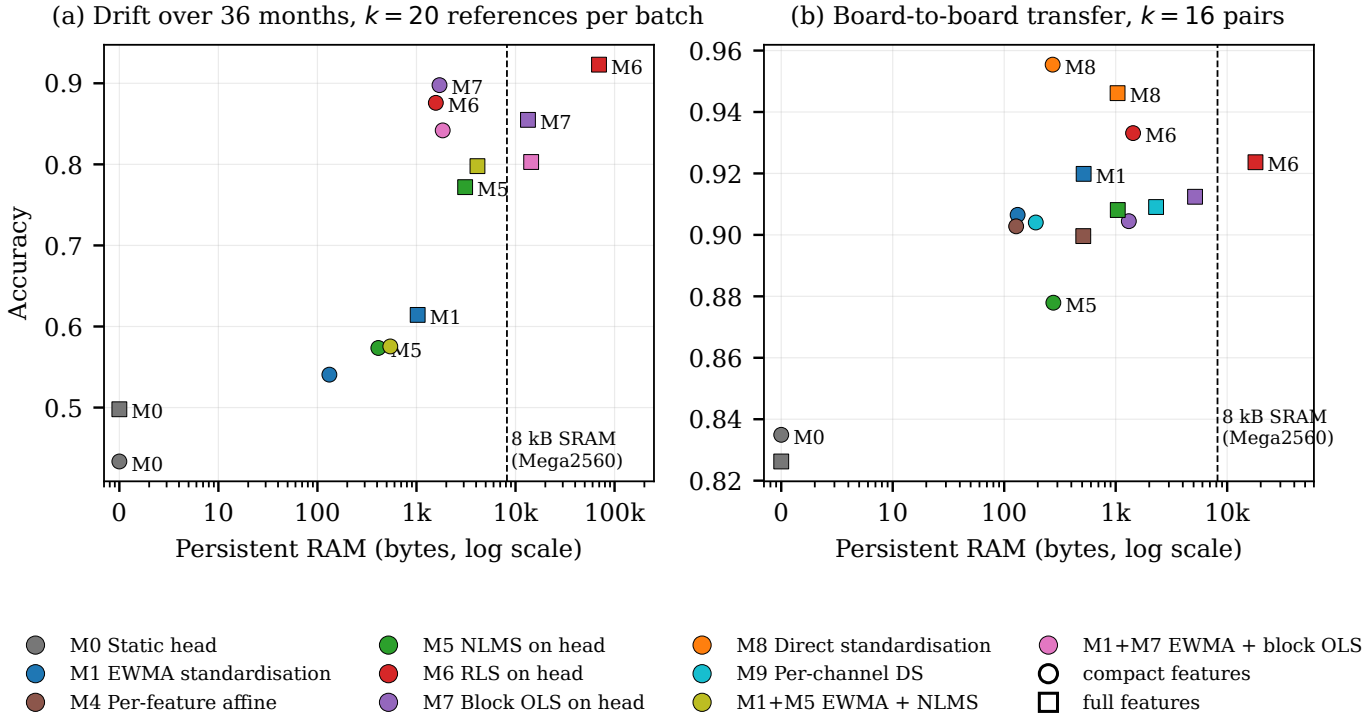


Fig. 1. Accuracy versus persistent RAM (adaptation state plus the head if the method modifies it; DS fused into the head). (a) D1 drift stream, batches 3–10, $k = 20$ references per batch. (b) D2 transfer, mean over 19 board pairs, $k = 16$ transfer pairs or references. Circles: compact features ($d = 16$); squares: full features ($d = 128$ in D1, $d = 64$ in D2). The static head needs no RAM beyond the feature vector and is drawn at 0. The dashed line marks the 8 kB SRAM of the Mega2560, which also has to hold the stack and the application.

are almost insensitive to the order, since their references still cover all gases. When, however, all references of a session come from one gas, as in a calibration with a single test gas, every label-driven update falls below the static head (0.434 and 0.498); block least squares falls to 0.136 at $d = 16$, because the refit drives the head towards the one class it sees. Labelled adaptation of the classifier, therefore, requires calibration sessions that cover the classes, and this requirement weighs more than the choice between NLMS, RLS and block least squares.

C. Implementation variants

The selected RLS forgetting factor $\beta = 0.9$ lies on the edge of the E2 grid. With $\beta = 0.95$, accuracy at $d = 128$ and $k = 20$ rises from 0.923 to 0.940, whereas at $d = 16$ it falls from 0.876 to 0.813; $\beta = 0.99$ gives 0.909 and 0.654. Bounding the trace of $\boldsymbol{P}$ (M6b) stabilises the update but lowers the accuracy at $d = 16$, $k = 20$ to 0.613. The edge selection therefore did not hide a large gain, and we keep the frozen values; the reported RLS figures depend on β by several points.

TABLE III
PAIRED DIFFERENCES IN ACCURACY

Comparison	Mean	Better	95% interval
E2, drift stream: units are the 8 test batches			
M1 vs M0, $d = 16$, $k = 0$	+0.107	6/8	[-0.09, +0.38][a]
M1 vs M0, $d = 128$, $k = 0$	+0.116	7/8	[-0.09, +0.26][a]
M5 vs M0, $d = 128$, $k = 20$	+0.274	8/8	[+0.17, +0.51][a]
M6 vs M5, $d = 128$, $k = 20$	+0.151	8/8	[+0.05, +0.29][a]
M7 vs M6, $d = 16$, $k = 20$	+0.022	4/8	[-0.05, +0.16][a]
M6 vs M5, $d = 16$, $k = 20$	+0.302	8/8	[+0.12, +0.47][a]
E4, transfer: units are the 19 board pairs			
M1 $k = 0$ vs M0 $k = 0$	+0.065	15/19	[+0.008, +0.138][b]
M8 $k = 16$ vs M1 $k = 16$	+0.049	16/19	[-0.004, +0.123][b]
M8 $k = 16$ vs M6 $k = 16$	+0.022	13/19	[+0.003, +0.045][b]
M8 $d = 16$ vs $d = 64$, $k = 16$	+0.009	10/19	[-0.013, +0.040][b]
M1 $k = 16$ vs M1 $k = 0$	+0.011	15/19	[-0.000, +0.020][b]
M8 $k = 8$ vs M1 $k = 8$	+0.025	11/19	[-0.006, +0.071][b]

[a]Range of the per-batch differences; an interval over the 20 reference draws would only describe this one stream and is given in the repository. [b]Percentile interval of a cluster bootstrap that resamples the five boards (10 000 replicates). Both comparisons in a row use identical test records.

TABLE IV
DRIFT STREAM UNDER LESS FAVOURABLE CALIBRATION ($k = 20$, ACCURACY, BATCHES 3–10)

		References of all gases		References of one gas	
Method	d	shuffled	gas blocks	shuffled	gas blocks
M1 EWMA	16	0.541	0.514	0.544	0.520
M5 NLMS	16	0.574	0.565	0.391	0.371
M6 RLS	16	0.876	0.867	0.329	0.313
M7 block OLS	16	0.898	0.890	0.136	0.134
M1+M5 EWMA + NLMS	16	0.575	0.554	0.424	0.396
M1 EWMA	128	0.614	0.541	0.615	0.538
M5 NLMS	128	0.772	0.752	0.472	0.462
M6 RLS	128	0.923	0.924	0.536	0.522
M7 block OLS	128	0.855	0.855	0.355	0.366
M1+M5 EWMA + NLMS	128	0.798	0.675	0.597	0.485

First column: protocol of E2. Gas blocks: within each batch the recordings arrive grouped by gas in a random gas order. One gas: all k references of a session come from one randomly chosen gas. 20 seeds; the 95% intervals over seeds are at most ± 0.036 and are given in the repository.

D. Concentration under drift

For regression (Table V), the static heads have an RMSE of 0.31–0.34 in $\log_{10}$ ppmv, i.e., a typical factor of about two in concentration. Without labels, EWMA standardisation helps at $d = 128$ (0.241) but hurts at $d = 16$ (0.352): within one gas, the incoming concentration mix differs from that of the source, and standardisation removes part of the concentration signal itself. With five references of the gas, NLMS at $d = 128$ reduces the RMSE to 0.154 with 4 B of state, and block least squares to 0.138. RLS with the selected forgetting factor $\beta = 0.8$, which lies on the edge of the grid, becomes unstable for toluene and ammonia at $k = 20$ (worst gas 0.95 at $d = 16$), the known covariance wind-up of RLS with strong forgetting and few informative samples. With $\beta = 0.95$ (post hoc), RLS is stable and reaches 0.171 at $d = 16$ and 0.124 at $d = 128$, with a worst gas of 0.27 and 0.19.

E. Calibration transfer between twin boards

Board-to-board variation is the more severe problem in D2 (Table VI, Fig. 1(b)). A master head applied to a slave board reaches 0.831 on average, with a worst pair of 0.65, whereas within a board the accuracy stays between 0.87 and 0.95 over

TABLE V
CONCENTRATION REGRESSION UNDER DRIFT: RMSE IN $\log_{10}$ PPMV (BATCHES 3–10, SIX GASES, 20 SEEDS)

	$d = 16$			$d = 128$		
Method	$k = 1$	5	20	$k = 1$	5	20
Static head	0.312	0.312	0.329	0.326	0.326	0.340
EWMA standardisation	0.352	0.353	0.364	0.241	0.241	0.249
NLMS on head	0.308	0.294	0.274	0.183	0.154	0.140
RLS on head	0.283	0.193	0.322	0.182	0.148	0.317
Block OLS on head	0.224	0.184	0.174	0.176	0.138	0.119

TABLE VI
CALIBRATION TRANSFER BETWEEN TWIN BOARDS: MEAN (WORST) ACCURACY OVER 19 BOARD PAIRS, $d = 16$

Method	$k = 0$	4	8	16	RAM (B)
M0 static	0.831 (0.65)	0.831 (0.66)	0.832 (0.67)	0.835 (0.70)	0
M1 EWMA	0.895 (0.71)	0.899 (0.70)	0.900 (0.71)	0.907 (0.71)	132
M4 affine	–	0.889 (0.74)	0.896 (0.73)	0.903 (0.73)	128
M9 chan. DS	–	0.891 (0.74)	0.898 (0.74)	0.904 (0.74)	192
M8 DS	–	0.878 (0.75)	0.925 (0.87)	0.955 (0.92)	272
M5 NLMS	–	0.845 (0.72)	0.855 (0.72)	0.878 (0.72)	276
M6 RLS	–	0.878 (0.72)	0.902 (0.74)	0.933 (0.85)	1 432
M7 block OLS	–	0.848 (0.70)	0.876 (0.74)	0.904 (0.79)	1 312
M8 DS, $d = 64$	–	0.890 (0.75)	0.919 (0.77)	0.946 (0.87)	1 040

k: transfer pairs (M4, M8, M9) or labelled slave references (M5–M7); M1 does not use them. RAM: bytes kept between samples; M8 is counted fused into the head ($4C(d+1)$ B), the separate map takes $4d(d+1)$ B.

up to four measurement days. EWMA standardisation of the slave stream, again without labels, raises transfer accuracy to 0.895 with 132 B; the gain of 0.065 has a board-bootstrap interval of $[0.008, 0.138]$. DS on the compact features reaches 0.925 with eight transfer pairs and 0.955 with sixteen, with a worst pair of 0.92, and it can be fused into the head, so that only 272 B are kept in RAM. Against EWMA evaluated on the same records, DS is better in 16 of 19 pairs by 0.049, but with five boards the interval $[-0.004, 0.123]$ does not exclude zero; against RLS on the head it is better by 0.022 $[0.003, 0.045]$. DS on all 64 features is not better (0.946). The paired affine and per-sensor corrections stay at 0.89–0.90, at the level of EWMA, which needs no transfer samples at all.

The transfer pairs are drawn at random from the 40 gas–level conditions, which spreads them over gases and concentrations; a practical transfer with one or two gases would be less favourable, as the one-gas scenario of D1 suggests. The result is also robust to the data concerns of D2. Without the methane recordings of board 5, a head trained on board 5 cannot recognise methane at all, so we compare on the 15 pairs whose master is another board: DS reaches 0.954 instead of 0.957, EWMA 0.926 instead of 0.931 and the static head 0.857 in both cases. Without the nine recordings with missing samples, no mean changes by more than 0.003, and the order of the methods is unchanged.

VI. EXECUTION ON MICROCONTROLLERS

A. Agreement with the host

All six E5 sessions (two per board) passed validation: each kernel reproduced the host predictions on every fixture sample, the checksums of the updated heads and of the DS map

TABLE VII
RAM OF THE FIRMWARE CONFIGURATIONS IN BYTES (FLOAT32)

Configuration	d	Head	State	Persist.	Scratch	Calib.	Peak
M0 Static head	16	–	–	0	64	–	64
M1 EWMA	16	–	132	132	64	–	196
M5 NLMS	16	408	4	412	64	–	476
M6 RLS	16	408	1 160	1 568	132	–	1 700
M7 Block OLS	16	408	1 300	1 708	64	2 044	3 816
M1+M5 EWMA + NLMS	16	408	136	544	64	–	608
M1+M7 EWMA + OLS	16	408	1 432	1 840	64	2 044	3 948
M0 Static head	128	–	–	0	512	–	512
M1 EWMA	128	–	1 028	1 028	512	–	1 540
M5 NLMS	128	3 096	4	3 100	512	–	3 612
M6 RLS	128	3 096	66 568	69 664	1 028	–	70 692*
M7 Block OLS	128	3 096	10 260	13 356	512	70 140	84 008*
M1+M5 EWMA + NLMS	128	3 096	1 032	4 128	512	–	4 640
M1+M7 EWMA + OLS	128	3 096	11 288	14 384	512	70 140	85 036*
M8 DS, map	16	–	1 088	1 088	128	3 204	4 420
M8 DS, fused	16	272	–	272	64	4 292	4 628
M8 DS, map	64	–	16 640	16 640	512	25 092	42 244*
M8 DS, fused	64	1 040	–	1 040	256	41 732	43 028*

Head: classifier in RAM because the method modifies it (otherwise it stays in flash). Persist.: persistent RAM, head plus state, kept between samples. Scratch: feature vector of the current sample (and the RLS gain or the DS output). Calib.: temporary memory at the end of a session (Gram matrix, solution, targets, transfer pairs). D1: $C = 6$, $k = 20$; D2: $C = 4$, $k = 16$. *Exceeds the 7 392 B of SRAM left by the firmware on the Mega2560.

TABLE VIII
COMPLETE STREAMS ON THE BOARDS (E7)

	Mega2560	ESP32-C3	ESP32-S3
Complete D1 streams	6	11	11
Predictions differing (all RLS)	1 of 79 710	6 of 146 135	6 of 146 135
DS pairs matching host accuracy	3/3	3/3	3/3
Static head, $d = 16$ (μs/sample)	5 545	510	27.5
RLS update, $d = 16$ (μs/ref.)	20 912	1 685	76.3
Block OLS refit, $k = 20$ (ms)	158	11.8	0.68
DS fit + fusion, 16 pairs (ms)	289	21.7	1.14
Fused DS head (μs/sample)	4 721	447	25.2
Extractor (μs/sample)	1 278	96.9	5.35
Peak stack (B)	425	848	1 380
Largest arena (B)	4 628	70 688	70 688

D1: seed 0, $k = 20$; Mega2560 runs the compact-feature configurations only. Times include feature preprocessing and are means over the complete streams; the stack peak covers the whole stack (Mega2560) or the Arduino loop task (ESP32).

differed from the host by at most 3×10^{-6} relative, and the extractor reproduced the host features exactly. RLS at $d = 128$ does not fit the Mega2560 and is reported there as skipped.

Because the fixtures are short, E6 repeated the complete streams with the same float32 C kernels on the host. Over 920 complete E2 streams, 65 of 12.3 million predictions differ from the float64 host, and no accuracy changes by more than 1.3×10^{-5}; the differences occur where two class scores are nearly equal, mostly in RLS. DS fitted in float32 and fused into the head agrees with the separate float64 map on all 86 880 predictions at $d = 16$ and on all but 2 of 86 880 at $d = 64$. The extractor gives bit-identical features on all 640 complete recordings (38.0 million samples).

On the boards themselves (E7, Table VIII), the complete E2 stream of seed 0 with $k = 20$ was run for every deployable configuration: six streams with compact features on the Mega2560 and eleven on each ESP32, 13 285 predictions per stream. All methods except RLS reproduced the host float32 predictions without exception. RLS differed in 1 of 13 285 predictions on the Mega2560, in 2 at $d = 16$ and in 4 at $d = 128$ on the ESP32 boards, again where two class scores nearly coincide; the target libraries round some float32 operations differently from the host. DS was fitted on the board from 16 transfer pairs and fused into the head, and the resulting accuracy on pairs $1 \to 3$, $1 \to 4$ and $1 \to 5$ equalled the host value on every board. The extractor, fed with three complete 600 s recordings (173 000 samples), returned features within 1.5×10^{-8} relative of the host (the difference stems from the decimal transfer of the result).

B. Memory

Table VII gives the RAM of the firmware configurations; the E7 firmware allocates its arena in the same layout, up to 4 B of alignment padding. On the Mega2560, the E7 firmware itself uses 800 B of static RAM outside its buffers, which leaves 7 392 B for the method and the stack. Every configuration with compact features fits, including the refit of block least squares (3.8 kB peak) and the DS fit (4.4 kB for the separate map, 4.6 kB with fusion). With 128 features, EWMA, NLMS and their combination fit (at most 4.6 kB), whereas RLS keeps 69.7 kB and block least squares already exceeds the SRAM between sessions (13.4 kB) because of its reference buffer. The kernels themselves use little stack: the deepest call chain, the ridge fit with its Cholesky solver, takes 123 B on the AVR, 256 B on the RISC-V and 160 B on the Xtensa core according to the compiler (without library functions). Measured over complete streams, including the Arduino core, interrupts and serial output, the whole stack never exceeded 425 B on the Mega2560; on the ESP32 boards the loop task used at most 848 B (C3) and 1 380 B (S3) of its 8 kB stack. The arenas reported by the E7 firmware at initialisation reach 4 628 B for DS with fusion and 70 688 B for RLS at $d = 128$, in agreement with Table VII.

C. Time per call

Table IX and Fig. 2 give the median time per call. The two sessions on each board differ by at most 0.13%, and so do the ten repetitions within a session; the largest spread belongs to the single block refit on the ESP32-S3 (796 versus 797 μs), where the resolution of the timer becomes visible.

Three observations follow from these measurements. First, all kernels are fast in relation to the measurement cycle of a MOX array, which lasts minutes. The slowest per-sample kernel on the Mega2560, classification with an NLMS update at $d = 128$, takes 75 ms, and a DS fit from 16 pairs takes 0.27 s and is executed once per calibration session; after fusion, DS costs no more than the static head. The streaming extractor, which must keep up with the 100 Hz sampling, occupies 12.7% of the Mega2560, 1.0% of the ESP32-C3 and 0.05% of the ESP32-S3.

Second, label-driven adaptation is cheap compared with inference itself. An NLMS update increases the time of a static classification by a factor of 1.5–1.8 on all boards, and RLS at $d = 16$ by a factor of 2.9–4.1. The preprocessing with one logarithm per feature dominates the static classification on processors without a floating-point unit.

Third, the ESP32-C3, which has no floating-point unit, is

TABLE IX
MEASURED TIME PER CALL ON THE BOARDS (MEDIAN OF 20 TIMINGS IN TWO SESSIONS, μS) AND HOST ACCURACY

Kernel	One call	Host accuracy	Mega2560	ESP32-C3	ESP32-S3
Static head, $d = 16$	classify	0.433	5 401	505.4	25.2
Static head, $d = 128$	classify	0.498	42 542	4 039	197.0
EWMA, $d = 16$	classify	0.541	7 791	662.6	36.7
EWMA, $d = 128$	classify	0.614	61 438	5 298	289.0
NLMS, $d = 16$	classify + update	0.574	9 603	829.7	37.3
NLMS, $d = 128$	classify + update	0.772	74 769	6 528	287.2
RLS, $d = 16$	classify + update	0.876	22 102	1 805	72.8
RLS, $d = 128$	classify + update	0.923	does not fit	57 904	2 114
Block OLS, $d = 16$	refit, $k = 20$	0.898	164 426	11 650	797.0
DS, $d = 16$	fit, 16 pairs	0.955	274 708	19 792	1 224
DS, $d = 16$	map + classify	0.955	9 412	826.0	35.2
Feature extractor	one sample	–	1 265	98.1	4.79

Kernels are timed on fixtures of real samples stored in flash (E5); host accuracy is that of the same configuration in E2 ($k = 20$) or E4 ($k = 16$). RAM of each configuration is given in Table VII.

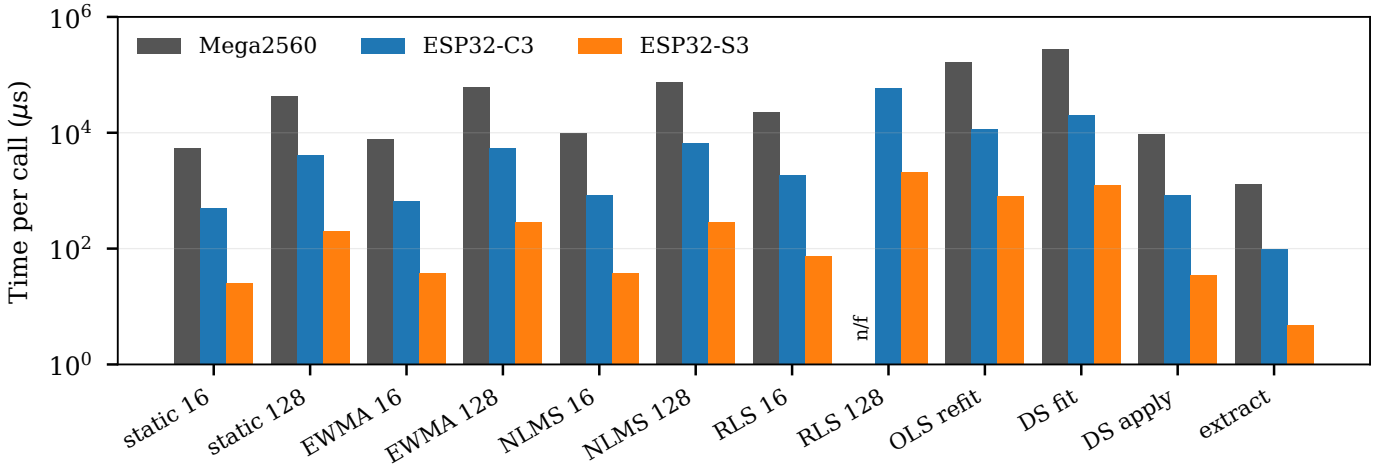


Fig. 2. Measured time per call on the three boards (median, log scale). "n/f": RLS at $d = 128$ does not fit the Mega2560. Classification and update kernels include feature preprocessing; block OLS and DS fit are executed once per calibration session.

10.7–14.1 times faster than the Mega2560, which is close to the ratio of their clock frequencies. The ESP32-S3 with a single-precision unit is a further 15–27 times faster. Since the instruction set, memory system and compiler also differ between the boards, these ratios describe the tested builds and do not isolate the effect of the floating-point unit.

VII. DISCUSSION

Taken together, the results give a deployment map. On an 8-bit microcontroller with 8 kB of SRAM, every method on compact features fits, and the choice depends on what references are available. Without any labels, EWMA standardisation is a reasonable first step: it costs 132 B, improved the drift stream in most batches and the transfer between boards on average, but it can lose accuracy in single batches and when gases arrive in long blocks. When a calibration session with about 20 known samples of all gases is possible, RLS or block least squares on 16 features give 0.88–0.90 on the 36-month stream with 1.6–1.7 kB of RAM. When the instrument is a copy of a calibrated master and 16 paired samples are available, DS fused into the head gives 0.955 with 272 B of persistent RAM. The full-feature variants of RLS, block least squares and DS do not fit the Mega2560 and require an ESP32-class device, where full-feature RLS costs 2.1 ms per sample on the S3 and 58 ms on the C3.

Time, in contrast, does not restrict the choice on any of the boards for the cycle times of MOX arrays. This conclusion is specific to slow chemical sensing. For faster sensors or for many arrays served by one processor, the measured times can be used directly to estimate the load.

Several limitations should be noted. The accuracy figures of E2 come from one 36-month stream of one instrument; the per-batch spread in Table III is a better guide to their uncertainty than intervals over seeds, and E4 rests on five boards. The scenarios of E6 show that the favourable calibration of E2 and the random transfer pairs of E4 overstate what label-driven updates achieve with single-gas sessions. The analyses of E6 and E7 were added after the protocol was frozen, and several hyperparameters lie on the edges of their grids. The head is a linear least-squares classifier, weaker than the ensembles used on the same data [6]. The planned Q15 replicas of the kernels are not included. Finally, each board type was represented by one unit, and energy consumption and worst-case execution time were not measured.

VIII. Conclusion

We have measured the accuracy of linear drift-compensation and calibration-transfer methods for MOX electronic noses together with their RAM and their execution time on three microcontrollers. The main findings are as follows.

1) Without labels, running standardisation of the features improved the static classifier on the 36-month drift stream from 0.498 to 0.614 with 1 028 B, in seven of eight test batches, and between twin boards from 0.831 to 0.895 with 132 B; its effect is not uniform and decreases when gases arrive in blocks.
2) With 20 references per batch from all gases, RLS on the classifier reaches 0.923 but keeps 69.7 kB in RAM; on compact features, RLS and block least squares reach 0.876 and 0.898 with 1.6–1.7 kB. With references of a single gas, all label-driven updates fall below the static classifier.
3) Between twin boards, direct standardisation on 16 compact features reaches 0.955 with 16 transfer pairs; fused into the classifier, it keeps 272 B in RAM and needs 4.6 kB during the fit.
4) The float32 kernels reproduce the host on complete streams, also when they run on the three boards, with at most four near-tie differences per 13 285 predictions for RLS; even on the Mega2560 the slowest per-sample kernel takes 75 ms, so for gas sensor arrays memory rather than time limits the choice of method.

The natural next step is to evaluate the methods with a physical sensor array and realistic calibration sessions, and to add fixed-point kernels and energy measurements.

Data and Code Availability

Protocol, source code, results, firmware sources, the raw board logs and the log of post hoc changes are available in the public repository [23], which will be archived with a DOI on acceptance. The E1/E2 protocol was frozen at commit `4a439fb`, E3/E4 at `c9799aa`; the firmware of E5 is at `578719c`, and its board captures and their analysis at `4e8adc1`; the complete-stream board captures of E7 are in `results/e7_20261110`. The datasets are available from the UCI repository [17], [18], [20] and are not redistributed.

Acknowledgment

Anthropic Claude was used to generate the code and to draft the text. The author reviewed, edited and verified all content and takes full responsibility for it.

References

[1] K. C. Persaud and G. H. Dodd, “Analysis of discrimination mechanisms in the mammalian olfactory system using a model nose,” *Nature*, vol. 299, no. 5881, pp. 352–355, 1982, doi: 10.1038/299352a0.

[2] S. Marco and A. Gutiérrez-Gálvez, “Signal and data processing for machine olfaction and chemical sensing: A review,” *IEEE Sensors J.*, vol. 12, no. 11, pp. 3189–3214, Nov. 2012.

[3] T. Artursson, T. Eklöv, I. Lundström, P. Mårtensson, M. Sjöström, and M. Holmberg, “Drift correction for gas sensors using multivariate methods,” *J. Chemometrics*, vol. 14, nos. 5–6, pp. 711–723, 2000.

[4] M. Padilla, A. Perera, I. Montoliu, A. Chaudry, K. Persaud, and S. Marco, “Drift compensation of gas sensor array data by orthogonal signal correction,” *Chemometr. Intell. Lab. Syst.*, vol. 100, no. 1, pp. 28–35, 2010, doi: 10.1016/j.chemolab.2009.10.002.

[5] A. Ziyatdinov, S. Marco, A. Chaudry, K. Persaud, P. Caminal, and A. Perera, “Drift compensation of gas sensor array data by common principal component analysis,” *Sens. Actuators B, Chem.*, vol. 146, no. 2, pp. 460–465, 2010, doi: 10.1016/j.snb.2009.11.034.

[6] A. Vergara, S. Vembu, T. Ayhan, M. A. Ryan, M. L. Homer, and R. Huerta, “Chemical gas sensor drift compensation using classifier ensembles,” *Sens. Actuators B, Chem.*, vols. 166–167, pp. 320–329, 2012, doi: 10.1016/j.snb.2012.01.074.

[7] L. Zhang and D. Zhang, “Domain adaptation extreme learning machines for drift compensation in E-nose systems,” *IEEE Trans. Instrum. Meas.*, vol. 64, no. 7, pp. 1790–1801, Jul. 2015.

[8] I. Rodriguez-Lujan, J. Fonollosa, A. Vergara, M. Homer, and R. Huerta, “On the calibration of sensor arrays for pattern recognition using the minimal number of experiments,” *Chemometr. Intell. Lab. Syst.*, vol. 130, pp. 123–134, 2014, doi: 10.1016/j.chemolab.2013.10.012.

[9] Y. Wang, D. J. Veltkamp, and B. R. Kowalski, “Multivariate instrument standardization,” *Anal. Chem.*, vol. 63, no. 23, pp. 2750–2756, 1991, doi: 10.1021/ac00023a016.

[10] J. Fonollosa, L. Fernández, A. Gutiérrez-Gálvez, R. Huerta, and S. Marco, “Calibration transfer and drift counteraction in chemical sensor arrays using direct standardization,” *Sens. Actuators B, Chem.*, vol. 236, pp. 1044–1053, 2016, doi: 10.1016/j.snb.2016.05.089.

[11] R. Li, Z. Li, B. A. Kofi, J. Sun, Y. He, and M. Jiao, “A comprehensive review of algorithms for drift compensation in metal oxide semiconductor gas sensor arrays,” *Chemosensors*, vol. 14, no. 6, Art. no. 143, 2026, doi: 10.3390/chemosensors14060143.

[12] A. Krayden, M. Avraham, H. Ashkar, T. Blank, S. Stolyarova, and Y. Nemirovsky, “TinyML-based real-time drift compensation for gas sensors using spectral–temporal neural networks,” *Chemosensors*, vol. 13, no. 7, Art. no. 223, 2025, doi: 10.3390/chemosensors13070223.

[13] T. Liu *et al.*, “Active learning on dynamic clustering for drift compensation in an electronic nose system,” *Sensors*, vol. 19, no. 16, Art. no. 3601, 2019, doi: 10.3390/s19163601.

[14] J. Cao, T. Liu, J. Chen, T. Yang, X. Zhu, and H. Wang, “Drift compensation on massive online electronic-nose responses,” *Chemosensors*, vol. 9, no. 4, Art. no. 78, 2021, doi: 10.3390/chemosensors9040078.

[15] F. Sun, R. Sun, and J. Yan, “Cross-domain active learning for electronic nose drift compensation,” *Micromachines*, vol. 13, no. 8, Art. no. 1260, 2022, doi: 10.3390/mi13081260.

[16] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, “On-device training under 256KB memory,” in *Adv. Neural Inf. Process. Syst.*, 2022. [Online]. Available: `https://arxiv.org/abs/2206.15472`

[17] A. Vergara, “Gas sensor array drift dataset,” UCI Machine Learning Repository, 2012. [Online]. Available: `https://archive.ics.uci.edu/dataset/224`

[18] A. Vergara, “Gas sensor array drift dataset at different concentrations,” UCI Machine Learning Repository, 2013. [Online]. Available: `https://archive.ics.uci.edu/dataset/270`

[19] M. K. Muezzinoglu, A. Vergara, R. Huerta, N. Rulkov, M. I. Rabinovich, A. Selverston, and H. D. I. Abarbanel, “Acceleration of chemo-sensory information processing using transient features,” *Sens. Actuators B, Chem.*, vol. 137, no. 2, pp. 507–512, 2009, doi: 10.1016/j.snb.2008.10.065.

[20] J. Fonollosa, “Twin gas sensor arrays,” UCI Machine Learning Repository, 2016, doi: 10.24432/C5MW3K.

[21] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2014.

[22] A. C. Davison and D. V. Hinkley, *Bootstrap Methods and Their Application*. Cambridge, U.K.: Cambridge Univ. Press, 1997.

[23] O. Kuznetsov, “enose-drift-mcu: Drift compensation and calibration transfer for electronic noses on microcontrollers,” GitHub, 2026. [Online]. Available: `https://github.com/KuznetsovKarazin/enose-drift-mcu`